

JUNCTIONX KOREA 2026 · LABLUP × FURIOSAAI —
BUILD THE ULTIMATE AGENT SQUAD

[541] BIBIMBAP

관측할 수 있으면 줄일 수 있다.

If you can observe it, you can reduce it.

팀 CouchPotato (54-1) · AI:GO 스쿼드 + Trace Viewer + 의인화 그림책



Conductor(직접 풀이) + Solver
전원 gpt-oss-120b on FuriosaAI RNGD

KEYNOTE QUESTION

“손에 쥘 수 있는 모델로 어디까지 갈 수 있는가?”

“How far can you go with a model you can actually hold in your hands?” — Lablup × FuriosaAI keynote

OUR ANSWER

낭비를 볼 수 있을 만큼 멀리 —
그리고 **그만큼 싸게.**

Far enough to see the waste — and exactly that much cheaper.

- 더 큰 두뇌가 아니라 더 적은 낭비가 접근성의 실체
- Trace를 보고 즐겼고, 재투자 실험으로 **채점 경로**까지 읽어냈다
- 그 과정 전부가 **리플레이**된다 — 꾸밈이 아니라 기록



로그를 Trace로 해부
26,808 tok

같은 문항 ÷ 17
1,611 tok

채점 경로 발견
→ 단순화

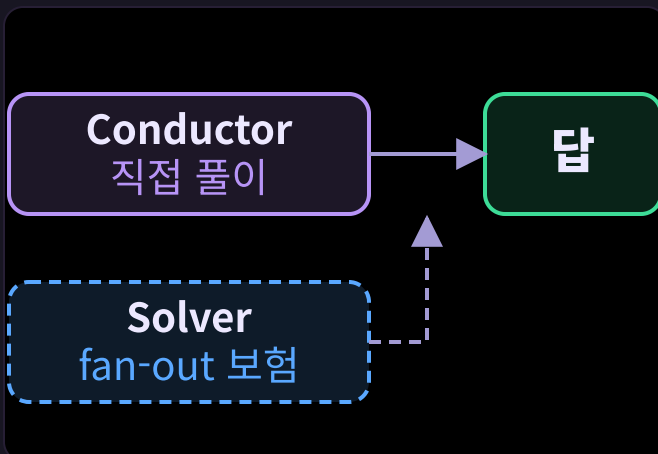
same item · math-visible-0001 · all numbers measured

WHAT WE BUILT

스쿼드 하나, 뷰어 하나, 그림책 하나

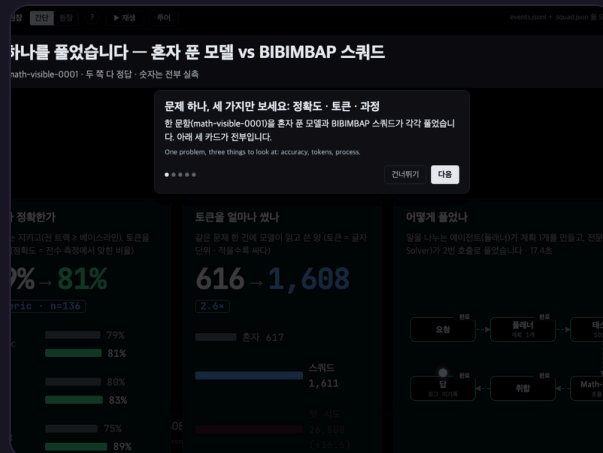
A squad on AI:GO, a Trace Viewer that reads the raw log, and a storybook for everyone else

스쿼드 Squad



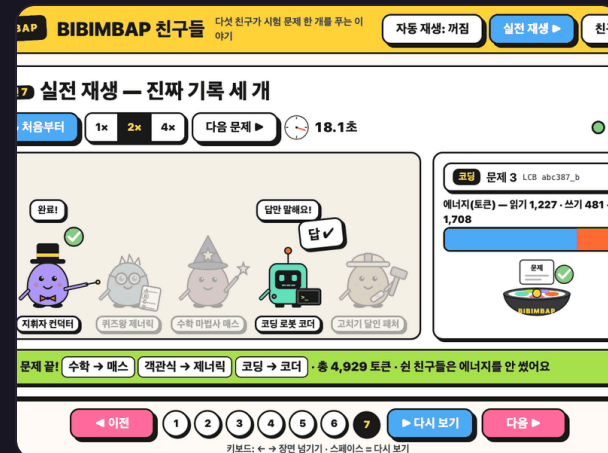
.squad.json + one-shot × 3 ·
AI:GO 스쿼드 위에 2 에이전트
(Conductor 직접 풀이 + Solver 쌍둥이), 전원 gpt-oss-120b · 전부 **측정**
로 결정

Trace Viewer viewer.html



간단 모드(스코어보드 + 투어) ↔
원장 모드(루브릭 6축 원장) · 원본
events.jsonl을 그대로 읽어
리플레이

그림책 kids.html



다섯 친구로 의인화한 에이전트가
진짜 기록으로 문제를 푸는 이야기
— 심사위원 아닌 누구에게도 설명
되도록

첫 스쿼드는 43배를 낭비했다

Same problem, both correct — our first squad burned $43\times$ the tokens of a single model

43×

토큰 기준 · 시간은 $12\times$ ($5.09\text{ s} \rightarrow 60.2\text{ s}$)

- 같은 문항 `math-visible-0001`, 둘 다 정답
- 단독 `gpt-oss-120b`: **617 tok** · **5.09 s**
- 스쿼드 v1 (8 에이전트): **26,808 tok** · **60.2 s**

단독 모델 `gpt-oss-120b`

 **617 tok** 5.09 s

BIBIMBAP 스쿼드 **v1** · 8 에이전트 · 3 태스크 · 28 호출


26,808 tok 60.2 s 막대 = 입력+출력 토큰, 같은 축

WHAT THE TRACE SHOWED

Trace를 열자 낭비는 세 층이었다

Three findings, all from the log — duplicated tasks, a spinning loop, and re-sent context

×3

플래너 중복

문제 1개를 같은 SOLVE 태스크 3개로 분해 — 비용도 3배

28회

루프 공회전

태스크당 11 · 6 · 11회 호출, 호출당 출력 평균 84 tok — 풀이 본문은 마지막 1회(315 tok)뿐

91%

비용 = 입력 재전송

입력이 487 → 1,277 tok으로 매 호출 누적 재전송 — 호출 1회 = 컨텍스트 1회 더

SOLVE #1 ● ● ● ● ● ● ● ● ● ● 11회

SOLVE #2 ● ● ● ● ● 6회

SOLVE #3 ● ● ● ● ● ● ● ● ● ● ● 11회

● 공회전 호출 (평균 84 tok)

● 답이 나온 1회 · 315 tok

26,808 tok 분해



■ 입력(컨텍스트 재전송) 91%

■ 출력 9%

→ 원칙 0: 불필요한 에이전트 호출 금지
(호출 1회 = 컨텍스트 1회 더)

AFTER THE TRACE · 8/22 16:54 → 19:27

로그를 보고 고쳤더니 ÷17

Same squad, same problem: 26,808 → 1,611 tokens, 60 s → 17 s, now 2.6× a solo model (was 43×)

- 루프 규약 역공학 — “EXACTLY ONE task” + PLAN READY, 빈 본문 금지 → 태스크 3→1, 호출 28→2
- 로그에 없던 비용 — 라우터 통계로 플래너 thinking 발견 (Qwen3 호출당 ~590 tok 출력) → /no_think 한 줄로 1,229 → 54 tok
- 남은 2.6×는 플래너 1홉 + 취합 = 관측의 고정 비용 — 낭비가 아니라 영수증

v1 첫 완주 · 8 에이전트 · 28 호출

26,808 tok · 60.2 s

v3.7b · 5 에이전트 · 2 호출

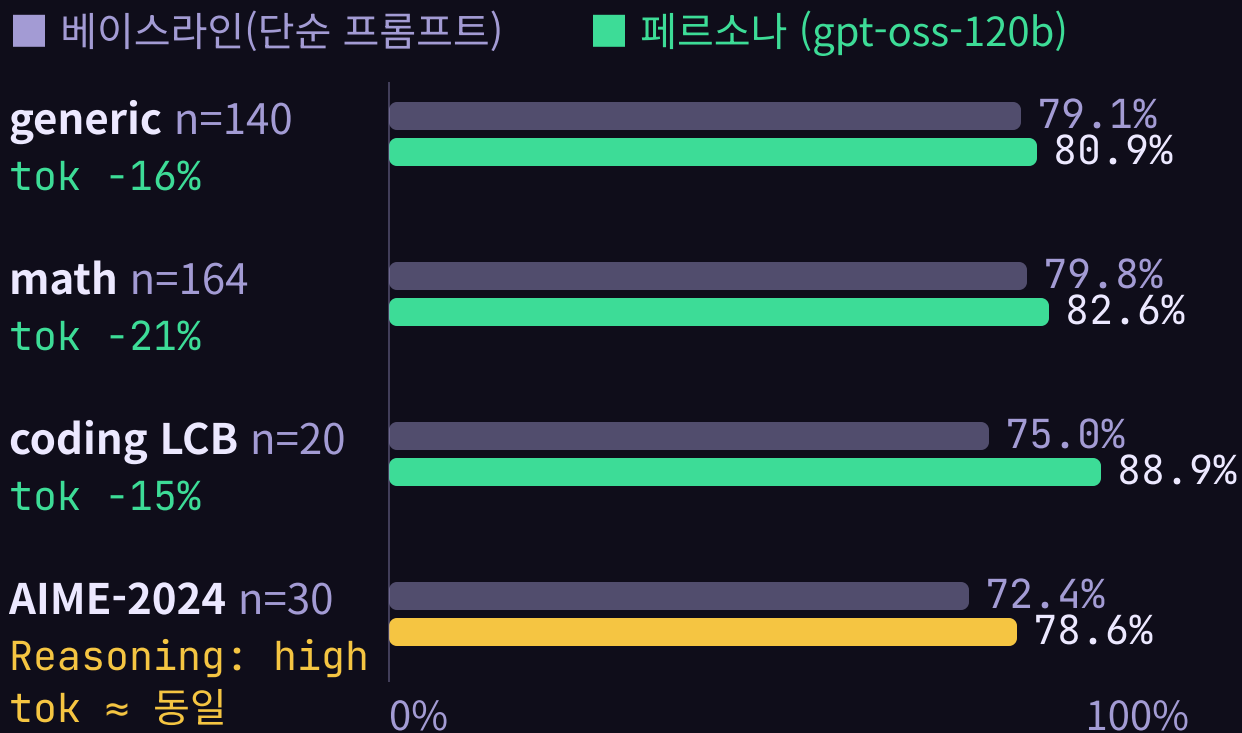
1,611 tok · 17 s · ÷17

단독 gpt-oss-120b (기준)

617 tok · 5.09 s · 스쿼드는 이 2.6×

정확도는 지키고, 출력 토큰은 -15~-21%

Full set, same n, same endpoint: simple prompt → persona. Models excluded by numbers, not assumptions



TRIED & EXCLUDED — MEASURED

모델	generic	AIME	LCB
gpt-oss-120b	80.9%	78.6%	88.9%
Qwen3-32B	66.9%	43.3%	35%
EXAONE-236B	69%	—	60%

EXAONE은 타임아웃 35-50% → 제외 · 세 모델 모두 같은 문항

정직한 정정: 20문항 베이스라인 “60%”는 표본 불운 — 전수 140에선 79.1%. 페르소나의 몫은 “+21%p”가 아니라 **정확도 유지 + 토큰 절감**.

전달률, 그리고 ‘누구의 마지막 말’이 채점되는가

Delivery rate — and whose last message gets graded. Score = weighted accuracy (coding .50 · math .25 · generic .25); request counts were the trace we read

우리 2차 · v3.5 thinking 플래너 · Qwen3 1.5회/문항

 0.186

우리 3차 · /no_think · gpt-oss 2.9회/문항

 0.254

우리 v7.1 · 앙상블 · 10.7회/문항 · 5.37M tok · 9 m 16 s

 0.045 coding 0 · math 0 · generic 17.9%

#3 DemoDayCare · gpt-oss 147회 = 1/문항 · Qwen3 112회

 0.393

#2 TheresNoFree · 1.9회/문항

 0.403

#1 MISHULTA · gpt-oss 단일 · 2.2회/문항

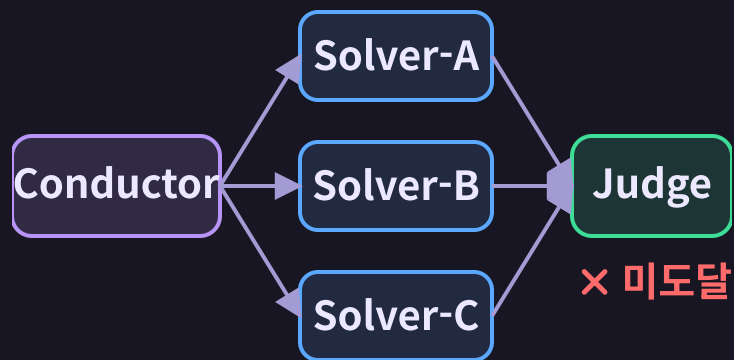
 0.426

- v7.1 앙상블 = 0.045 — 로컬에선 정답을 내던 Judge의 답이 채점기에 **달지 않았다**
- 증거는 요청 수: DemoDayCare의 gpt-oss가 정확히 147회 = 1/문항, Qwen3는 112회 — Qwen3가 매 문항 플래너였다면 불가능 → **플래너가 직접 답한다**
- 결론: 채점기는 계획 단계 플래너의 최종 메시지를 읽는다. thinking 플래너는 스스로 답해 0.186·0.254, “PLAN READY”를 완벽히 지킨 v7.1은 0.045

앙상블은 옳았고, 채점 경로가 달랐다 → 최종: 플래너가 직접 푼다

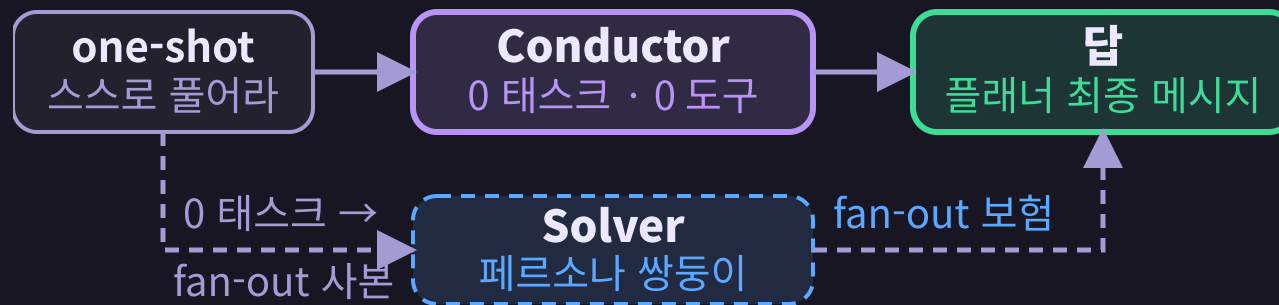
The ensemble was right; the grading path was not. Final: the planner answers directly in the planning phase

실험 · V7.1 앙상블 → 0.045



Judge의 정답은 태스크 결과에만 있었고, 채점기는 플래너의 마지막 말(“PLAN READY”)을 읽었다. 로컬 완주(12,544 tok · 37 s)는 정답이었다.

최종 · V6.0 DIRECT · 2 에이전트 · 전원 GPT-OSS-120B



- 채점 = 계획 단계 플래너의 최종 메시지 → Conductor가 답만 쓴다
- 상위 3팀 공통: gpt-oss ≈ 1회/문항 · Reasoning: high · ≈1.5K tok/문항 (추정)
- 러너 충실 selfeval: generic 76.0% (100문항 채점 · 재측정 중)

리더보드 최종: __ (결과 대기 · 편집)

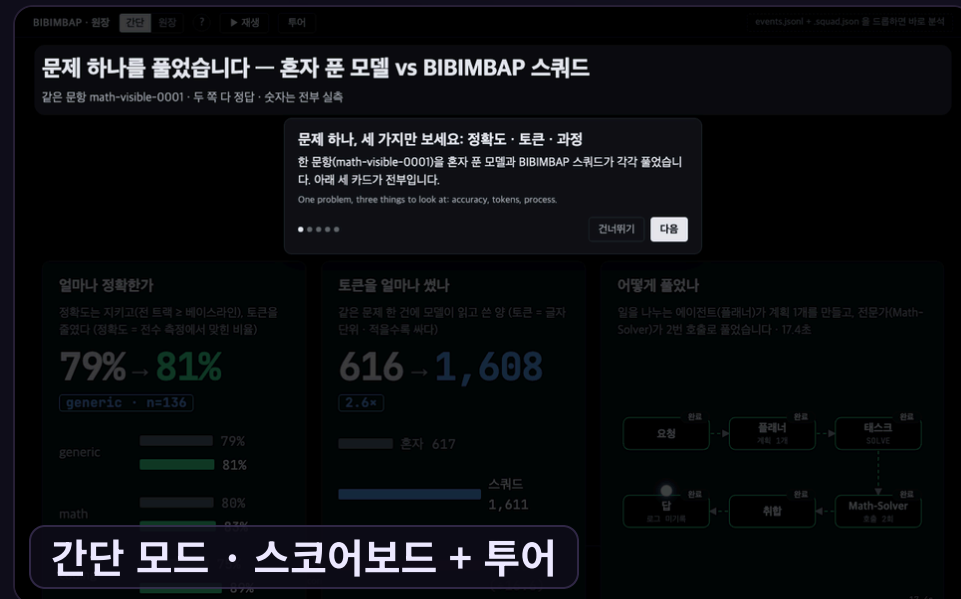
VISUALIZATION · 30점 · 루브릭 6축 ↔ 화면 요소 1:1

루브릭 여섯 단어를 화면 요소로 번역했다

Each rubric axis maps to one viewer element; the viewer replays the raw AI:GO log, the storybook retells it

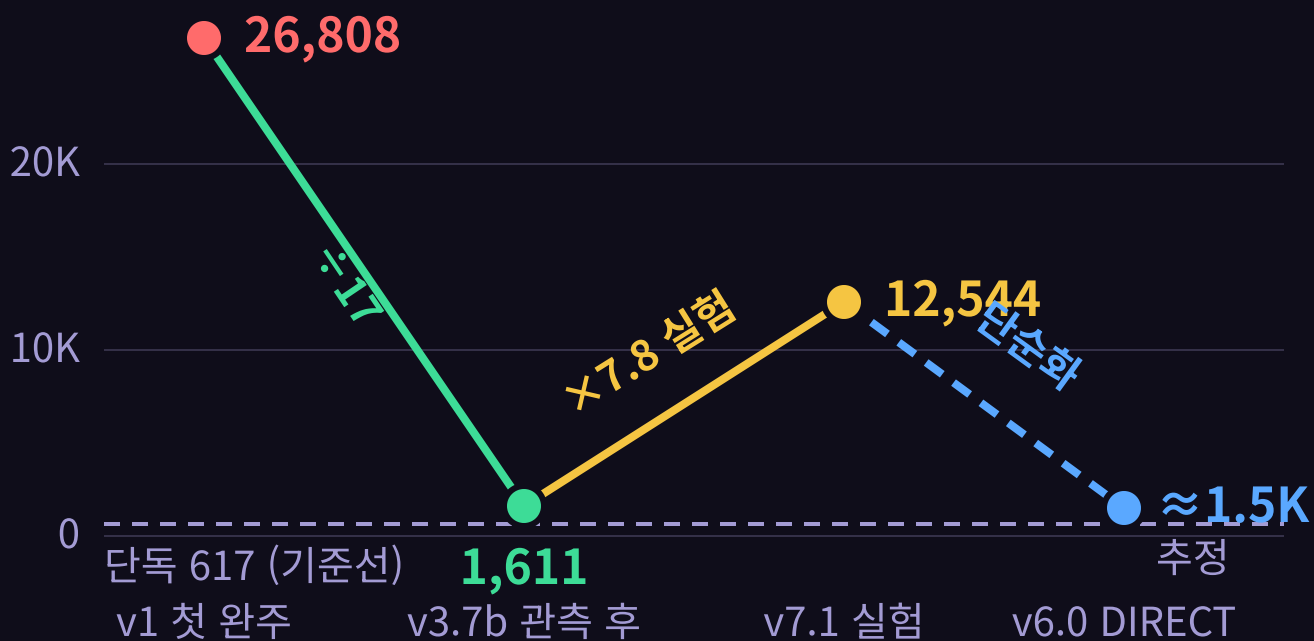
- **Observability** 스크러버 · 톱니 막대 · 유휴 레인
- **Interpretability** 캡션 · 상호작용 스트립 · 소계 줄
- **Traceability** 원장 · 원본 탭 · ← 로그 #k 링크
- **Explainability** 판단 카드 · 주장 카드 (규칙 도출)
- **Clarity** 한 숫자 + 한 문장 · 0 기준 막대
- **Insightfulness** 레이스 레인 · SPLIT · 캘리브레이션 곡선

로그에 없는 것도 보여준다: 솔버의 답 텍스트는 AI:GO 로그에 기록되지 않는다 — 뷰어는 이를 “로그 미기록”으로 표시한다.



관측 → 절감 → 재투자 실험 → 다시 관측 → 단순화

Tokens per item, same math problem: observe, cut, reinvest as an experiment, observe again (the public board), simplify



- 절감은 규칙이 됐다 — 비용의 91%가 입력 재전송 → 원칙 0 “불필요한 에이전트 호출 금지”
- 재투자 실험이 채점 경로를 드러냈다 — 답은 Judge에 있었지만, 채점기는 플래너의 마지막 말을 읽었다 (0.045)
- 다시 관측 → 단순화 — 공개 보드의 요청 수가 Trace였다: 상위 3팀 gpt-oss ≈ 1회/문항 → 플래너가 직접 푼다, ≈1.5K tok/문항 (추정)

CLOSING

관측할 수 있으면 줄일 수 있다 —
그리고 어디에 다시 쓸지 알 수 있다.

If you can observe it, you can reduce it — and know where to reinvest.

REPO github.com/jsonpassion/bibimbap · MIT · viewer ·
kids · traces · normalize.py

DEMO jsonpassion.github.io/bibimbap/ · 파일 하나, 외부
의존성 0, 로그를 드롭하면 재생



팀 CouchPotato (54-1) · [팀원 이름 — 편집 필요]